

RA Nutrition & Lifestyle App

Week-by-week implementation roadmap

Phase	Focus	Weeks
V1	Rule-based recommendations + symptom tracking + analytics dashboard	1–9
V2	Feedback loops + trigger detection + personalised insights	10–14
V3	ML personalization: flare-risk prediction + pain score forecasting	15–20+

Medical boundary: This is a supportive lifestyle tool — not a diagnostic system or a replacement for a rheumatologist or dietitian. Always display this boundary clearly in the app.

Recommended Tech Stack

Frontend	React + Vite, React Query, Recharts, Tailwind CSS
Backend	FastAPI (Python), Pydantic, SQLAlchemy ORM, Alembic migrations
Database	PostgreSQL — all user data, foods, meals, logs, recommendations
Nutrition data	USDA FoodData Central API (free) + your own curated meal table
ML layer (V3)	scikit-learn, XGBoost, pandas for feature engineering
Deployment	Render.com or Railway (free tier for MVP), Docker, Vercel/Netlify for frontend

V1 — Rule-Based MVP (Weeks 1–9)

Build the working product: rule-based food recommendations, symptom and meal logging, and a weekly analytics dashboard. No ML needed yet.

Week 1 — Problem Framing

V1

- Write user stories for meal logging, symptom tracking, next-meal suggestion, and dashboard views
- Define the medical safety boundary: supportive lifestyle tool, not a diagnostic system
- Define what the app is NOT (no diagnosing, no replacing clinicians)
- List every MVP screen and API endpoint with acceptance criteria

Deliverable

:

Requirements document + product flow diagram

Week 2 — Database Design

V1

- Design full ER diagram: users, foods, meals, meal_ingredients, daily_food_logs, symptom_logs, activity_logs, recommendation_logs, feedback_logs
- Choose tech stack: FastAPI (Python) + PostgreSQL + React
- Set up project repository, dev environment, and folder structure
- Write Alembic migration files for all initial tables

Deliverable

:

ER diagram + repo scaffolding with migrations

Week 3 — Nutrition Data Ingestion

V1

- Register for USDA FoodData Central API (free, no billing needed for low usage)
- Write Python script to pull and normalize food records (calories, protein, carbs, sugar, fiber, sodium, fat, omega-3)
- Load 500–1,000 common foods into PostgreSQL foods table
- Build /api/foods/search endpoint in FastAPI with filters
- Tip: Download the bulk 'foundation foods' CSV for fastest loading

Deliverable

:

Food master dataset loaded + /search endpoint working

Week 4 — Curated Meal Library

V1

- Create 100–200 curated meals manually (breakfast, lunch, dinner, snacks)
- Tag each meal: high-protein, low-sugar, flare-friendly, vegetarian, budget-friendly, quick-prep, etc.
- Assign anti-inflammatory score to each meal (0–10 scale)
- Map each meal to its ingredients with gram-level portion sizes
- Tip: Build a simple admin UI or spreadsheet importer to speed this up

Deliverable
: **meals + meal_ingredients tables seeded with 100–200 entries**

Week 5 — Rule Engine (Core Recommendation Logic)

V1

- Build meal scoring function: bonuses for fiber, omega-3, lean protein, calcium/vit-D; penalties for sugar, sodium, saturated fat, ultra-processed
- Add hard exclusion filters: allergy conflicts, user preference flags (vegetarian etc.)
- Add daily balancing: 'breakfast was low protein → boost lunch score for protein-rich meals'
- Add flare-mode logic: prefer simple, easy-to-prepare, joint-friendly meals
- Generate explanation text per recommendation: 'Suggested because today protein is low'

Deliverable
: **Working /api/recommend endpoint with explanation text**

Week 6 — Backend APIs

V1

- Auth endpoints: POST /register, POST /login, JWT token refresh
- Meal log endpoints: POST /logs/meals, GET /logs/meals/today
- Symptom log endpoints: POST /logs/symptoms (pain 0–10, fatigue, flare, stiffness, sleep)
- User profile + preferences endpoint: GET/PUT /users/me
- Dashboard summary endpoint: GET /dashboard/weekly (nutrient totals, pain avg, adherence)

Deliverable
: **Full backend API, testable via FastAPI auto-docs at /docs**

Week 7 — Frontend Screens (React)

V1

- Onboarding flow: name, age, weight, allergies, preferences, medications
- Home dashboard: today's recommendation card + quick log buttons
- Meal log screen: food search, add meal, see today's running nutrition totals
- Symptom screen: pain/fatigue/sleep sliders, flare toggle
- Recommendation card: meal image placeholder, name, score, explanation text, accept/skip buttons

Deliverable
: **Usable UI prototype connected to the real API**

Week 8 — Analytics Dashboard

V1

- Weekly trend charts: pain score, fatigue, meal quality score over 7 days (Recharts)
- Adherence summary: percentage of recommendations accepted vs skipped
- Simple correlation highlights: 'High-sugar days correlated with worse next-day pain'
- Top foods logged this week card

Deliverable
: **Analytics dashboard with charts and trend summaries**

Week 9 — Testing + MVP Release

V1

- Test edge cases: empty logs, new user with no history, full flare mode, allergy conflicts
- Manually review 20–30 recommendation outputs for quality
- Add medical disclaimer banners and escalation prompts for severe symptoms
- Deploy backend to Render.com or Railway (free tier works for MVP)
- Deploy frontend to Vercel or Netlify

Deliverable
: **Stable MVP deployed and live — end of V1**

V2 — Analytics & Feedback (Weeks 10–14)

Collect real user logs. Surface meaningful patterns and correlations. Make recommendations smarter using feedback without touching ML.

Week 10 — User Data Collection & Feedback Loops

V2

- Add skip/accept feedback logging to every recommendation (with optional 'why I skipped' note)
- Improve logging UX: reduce taps to log a meal, add quick-add shortcuts
- Start accumulating 90–180 daily logs per user — this is the ML fuel
- Track which meals are consistently skipped and build a per-user penalty

Deliverable
: **Rich longitudinal dataset building up + behavior-aware scoring**

Week 11 — Feedback-Aware Recommendation Improvement

V2

- Penalize consistently-skipped foods per user
- Boost score of accepted meal types per user
- Add 'foods this user rarely eats' penalty to stop recommending the same ignored meals

Deliverable
: **Per-user feedback-weighted recommendation engine**

Week 12 — Trigger Detection & Personalised Insights

V2

- Compute 3-day sugar average vs next-day pain score correlation
- Compute sleep hours vs next-day fatigue correlation
- Compute processed-food days vs next-day symptom severity
- Add weekly 'Patterns we noticed' card to the dashboard
- Example insight: 'On days after poor sleep, your pain score averages 2 pts higher'

Deliverable
: **Personalised weekly insights card shown in the app**

Week 13–14 — Seasonal Adjustment & V2 Polish

V2

- Add seasonal meal adjustment: lighter meals in summer, warming meals in winter
- Export weekly summary as PDF (optional, good for sharing with a rheumatologist)
- UX improvements based on any real user feedback
- Prep data quality audit ahead of ML: check missing values, label consistency

Deliverable
: **V2 stable + ML-ready dataset beginning to form**

V3 — ML Personalization (Weeks 15–20+)

Only after 90–180 daily logs per user: add personalized flare-risk prediction and pain score forecasting using interpretable ML models.

Week 15 — ML Dataset Preparation

V3

- Clean missing values in symptom and meal logs (impute or drop)
- Engineer lag features: 3-day sugar average, sleep last night, steps yesterday, flares in last 7 days
- Define prediction targets: pain score tomorrow (regression), flare tomorrow (binary classification)
- Create time-aware train/validation split — no data leakage across time
- Requirement: at least 90–180 daily logs per active user before starting

Deliverable
: **ML-ready feature dataset with engineered lag features**

Week 16–17 — Baseline ML Models

V3

- Logistic regression for flare risk (binary classification) — start simple
- Random forest or XGBoost for pain score prediction (regression)
- Evaluate: MAE and RMSE for pain regression, F1 + ROC-AUC for flare classification
- Compare ML vs rule-based baseline — only ship if ML meaningfully outperforms
- Add feature importance: 'Recent sleep pattern is the strongest predictor'

Deliverable

:

First personalization model + evaluation report

Week 18–20 — ML Integration & Monitoring

V3

- Expose prediction model as a FastAPI endpoint: POST /predict/flare-risk
- Show 'Tomorrow looks like a higher-risk day' card in the UI with reason
- Use prediction to adjust meal suggestions: on high-risk days, prioritise anti-inflammatory, easy-prep meals
- Add model monitoring: track prediction accuracy over time, retrain periodically
- Keep medical disclaimer prominent: this is a supportive tool, not a clinical prediction

Deliverable

:

Live personalised flare prediction integrated in the app — end of V3

Key reminders

- Do not add ML (V3) until you have 90–180 daily logs per active user — sparse data makes ML look impressive but perform poorly.
- The recommendation_logs table (what was shown, accepted, skipped, and why) is critical. Set it up in Week 2 and never skip logging it.
- The curated meal library (Week 4) is the hardest manual task. Build an admin UI early to make adding meals easier.
- Use USDA FoodData Central's bulk CSV download for Week 3 — faster than calling the API per food.
- Always show the medical disclaimer: consult a rheumatologist or dietitian for medical nutrition issues.